# R ↔ Python Cheat Sheet

*Working with Biological Data in Python and R* · same task, both languages, side by side

## 0 · Getting Started & Basics

| Task | R | Python |
|---|---|---|
| **Comment** | # a comment | # a comment |
| **Assign a value** | x <- 5 (or x = 5) | x = 5 |
| **Print / show** | print(x); cat(x, "\n") | print(x) |
| **Get help** | ?mean help(mean) | help(mean) # or mean? |
| **Install a package** | install.packages("dplyr") | pip install pandas |
| **Load a library** | library(dplyr) | import pandas as pd |
| **Working directory** | getwd(); setwd("path") | import os; os.getcwd() |
| **Check version** | R.version.string | import sys; sys.version |
| **Sequence 1..10** | 1:10 seq(1, 10) | range(1, 11) # 1..10 |
| **Concatenate / combine** | c(1, 2, 3) | [1, 2, 3] |

## 1 · Variables & Data Types

| Task | R | Python |
|---|---|---|
| **Number / integer** | x <- 3.14; n <- 5L | x = 3.14; n = 5 |
| **Text (string)** | s <- "gene" | s = "gene" |
| **Logical / boolean** | TRUE, FALSE | True, False |
| **Missing value** | NA | None / np.nan |
| **Check type** | class(x); is.numeric(x) | type(x); isinstance(x, float) |
| **Convert type** | as.numeric("5"); as.character(5) | float("5"); str(5) |
| **String length** | nchar(s) | len(s) |
| **Upper / lower** | toupper(s); tolower(s) | s.upper(); s.lower() |
| **Paste / join strings** | paste0("chr", 1) | f"chr{1}" # or "chr"+str(1) |
| **Substring** | substr(s, 1, 3) | s[0:3] |

## 2 · Data Structures

| Task | R | Python |
|---|---|---|
| **Vector / list** | v <- c(2, 4, 6) | v = [2, 4, 6] |
| **Index (1- vs 0-based)** | v[1] # first element | v[0] # first element |
| **Slice** | v[2:3] | v[1:3] |
| **Named map** | list(a=1, b=2) | {"a": 1, "b": 2} |
| **Sequence of numbers** | seq(0, 1, by=0.1) | np.arange(0, 1.1, 0.1) |

| Task | R | Python |
|---|---|---|
| **Data table** | data.frame(g=c("a"), x=1:2) | pd.DataFrame({"g":["a"], "x":[1,2]}) |
| **Factor / category** | factor(c("lo","hi")) | pd.Categorical(["lo","hi"]) |
| **Length / size** | length(v) | len(v) |
| **Unique values** | unique(v) | set(v)  # or  pd.unique(v) |
| **Apply over elements** | sapply(v, sqrt) | [math.sqrt(i) for i in v] |

## 3 · Reading & Writing Data Ch 3–4

| Task | R | Python |
|---|---|---|
| **Read CSV** | df <- read.csv("data.csv") | df = pd.read_csv("data.csv") |
| **Read TSV / delim** | read.delim("data.tsv") | pd.read_csv("data.tsv", sep="\t") |
| **Read Excel** | readxl::read_excel("d.xlsx") | pd.read_excel("d.xlsx") |
| **Write CSV** | write.csv(df, "out.csv") | df.to_csv("out.csv", index=False) |
| **Read with no header** | read.csv("d.csv", header=FALSE) | pd.read_csv("d.csv", header=None) |
| **FASTA / sequences** | Biostrings::readDNAStringSet(f) | Bio.SeqIO.parse(f, "fasta") |

## 4 · Inspecting a Dataset Ch 4–5

| Task | R | Python |
|---|---|---|
| **First / last rows** | head(df);  tail(df) | df.head();  df.tail() |
| **Dimensions** | dim(df);  nrow(df); ncol(df) | df.shape |
| **Column names** | names(df);  colnames(df) | df.columns |
| **Structure / dtypes** | str(df) | df.info();  df.dtypes |
| **Quick summary** | summary(df) | df.describe() |
| **Count categories** | table(df$group) | df["group"].value_counts() |
| **Unique values in col** | unique(df$group) | df["group"].unique() |

## 5 · Cleaning Data Ch 5

| Task | R | Python |
|---|---|---|
| **Find missing** | is.na(df) | df.isna() |
| **Count missing per col** | colSums(is.na(df)) | df.isna().sum() |
| **Drop missing rows** | na.omit(df) | df.dropna() |
| **Fill missing** | df[is.na(df)] <- 0 | df.fillna(0) |
| **Rename a column** | names(df)[1] <- "id" | df.rename(columns={"a":"id"}) |
| **Drop duplicates** | df[!duplicated(df), ] | df.drop_duplicates() |
| **Change a column type** | df$x <- as.numeric(df$x) | df["x"] = df["x"].astype(float) |
| **Trim whitespace** | trimws(s) | s.strip() |

| Task | R | Python |
|---|---|---|
| **Replace in string** | gsub("a", "b", s) | s.replace("a", "b") |

## 6 · Manipulating Data dplyr / pandas

| Task | R | Python |
|---|---|---|
| **Select columns** | select(df, x, y) | df[["x", "y"]] |
| **Filter rows** | filter(df, x > 5) | df[df["x"] > 5] |
| **New / modified column** | mutate(df, z = x + y) | df["z"] = df["x"] + df["y"] |
| **Sort** | arrange(df, desc(x)) | df.sort_values("x", ascending=False) |
| **Group + summarise** | df \|> group_by(g) \|><br>summarise(m = mean(x)) | df.groupby("g")["x"].mean() |
| **Pipe / chain** | df \|> filter(...) \|> select(...) | (df.query(...).filter(...)) |
| **Join / merge** | inner_join(a, b, by="id") | a.merge(b, on="id") |
| **Long ↔ wide** | pivot_longer(); pivot_wider() | df.melt(); df.pivot() |

## 7 · Summary Statistics Ch 6, 10

| Task | R | Python |
|---|---|---|
| **Mean / median** | mean(x); median(x) | x.mean(); x.median() |
| **SD / variance** | sd(x); var(x) | x.std(); x.var() |
| **Min / max / range** | min(x); max(x); range(x) | x.min(); x.max() |
| **Quantiles** | quantile(x, 0.25) | x.quantile(0.25) |
| **Sum / count** | sum(x); length(x) | x.sum(); x.count() |
| **Correlation** | cor(x, y) | x.corr(y) # or np.corrcoef |
| **Row/col means (matrix)** | rowMeans(m); colMeans(m) | m.mean(axis=1); m.mean(axis=0) |
| **Scale (z-score)** | scale(x) | (x - x.mean()) / x.std() |

## 8 · Plotting Ch 8–9

| Task | R | Python |
|---|---|---|
| **Library** | library(ggplot2) | import matplotlib.pyplot as plt<br>import seaborn as sns |
| **Scatter plot** | ggplot(df, aes(x,y)) +<br>geom_point() | plt.scatter(df.x, df.y)<br>sns.scatterplot(data=df, x="x", y="y") |
| **Line plot** | geom_line() | plt.plot(x, y) |
| **Histogram** | geom_histogram() | plt.hist(x) # sns.histplot(x) |
| **Boxplot** | geom_boxplot() | sns.boxplot(data=df, x="g", y="v") |
| **Bar chart** | geom_bar(stat="identity") | plt.bar(cats, vals) |
| **Colour by group** | aes(color = group) | hue="group" (seaborn) |
| **Labels / title** | labs(title="T", x="X") | plt.title("T"); plt.xlabel("X") |

| Task | R | Python |
|---|---|---|
| **Save figure** | ggsave("f.png") | plt.savefig("f.png", dpi=150) |

## 9 · Statistical Tests Ch 11, 13

| Task | R | Python |
|---|---|---|
| **t-test (2 groups)** | t.test(x ~ group, data=df) | scipy.stats.ttest_ind(a, b) |
| **Paired t-test** | t.test(x, y, paired=TRUE) | scipy.stats.ttest_rel(x, y) |
| **One-way ANOVA** | summary(aov(y ~ g, df)) | scipy.stats.f_oneway(a, b, c) |
| **Wilcoxon / Mann-Whitney** | wilcox.test(x ~ g) | scipy.stats.mannwhitneyu(a, b) |
| **Chi-square** | chisq.test(table(a, b)) | scipy.stats.chi2_contingency(t) |
| **Correlation test** | cor.test(x, y) | scipy.stats.pearsonr(x, y) |
| **Shapiro (normality)** | shapiro.test(x) | scipy.stats.shapiro(x) |
| **Adjust p-values (FDR)** | p.adjust(p, "BH") | statsmodels ... multipletests(p) |

## 10 · Regression & Models Ch 12

| Task | R | Python |
|---|---|---|
| **Linear model** | fit <- lm(y ~ x, data=df) | smf.ols("y ~ x", df).fit() |
| **Model summary** | summary(fit) | fit.summary() |
| **Coefficients** | coef(fit) | fit.params |
| **Predictions** | predict(fit, newdata) | fit.predict(new) |
| **Multiple predictors** | lm(y ~ x1 + x2, df) | smf.ols("y ~ x1 + x2", df).fit() |
| **Logistic regression** | glm(y ~ x, family=binomial) | smf.logit("y ~ x", df).fit() |
| **Residuals** | resid(fit) | fit.resid |

## 11 · Sequences & Genomics Ch 14-15

| Task | R | Python |
|---|---|---|
| **Library** | library(Biostrings) | from Bio.Seq import Seq |
| **Make a sequence** | DNAString("ATGC") | Seq("ATGC") |
| **Length** | nchar(seq);  width(seq) | len(seq) |
| **Reverse complement** | reverseComplement(seq) | seq.reverse_complement() |
| **Transcribe / translate** | translate(seq) | seq.translate() |
| **GC content** | letterFrequency(s,"GC") | gc_fraction(seq)  # Bio.SeqUtils |
| **Count a base** | countPattern("A", seq) | str(seq).count("A") |
| **Read FASTA** | readDNAStringSet("f.fa") | SeqIO.parse("f.fa", "fasta") |

## 12 · Machine Learning Ch 20 · scikit-learn / tidymodels

| Task | R | Python |
|---|---|---|
| **Train / test split** | initial_split(df, prop=0.7) | train_test_split(X, y, test_size=.3) |
| **Scale features** | step_normalize(all_numeric()) | StandardScaler().fit_transform(X) |
| **Fit a classifier** | fit(workflow, train) | clf.fit(X_train, y_train) |
| **Predict** | predict(fit, test) | clf.predict(X_test) |
| **Accuracy** | accuracy(pred, truth, est) | accuracy_score(y_test, y_pred) |
| **Confusion matrix** | conf_mat(pred, truth, est) | confusion_matrix(y_test, y_pred) |
| **Cross-validation** | fit_resamples(wf, vfold_cv(df)) | cross_val_score(clf, X, y, cv=5) |

## 13 · Reproducibility & Git Ch 19

| Task | R | Python |
|---|---|---|
| **Set a random seed** | set.seed(42) | np.random.default_rng(42) |
| **Record environment** | sessionInfo() | pip freeze > requirements.txt |
| **Relative paths** | here::here("data","x.csv") | Path(__file__).parent / "x.csv" |
| **Notebook** | R Markdown (.Rmd) → Knit | Jupyter (.ipynb) → Run All |
| **Git: snapshot** | — | git add . && git commit -m "msg" |
| **Git: back up** | — | git push / git pull |

**Note on indexing:** *R counts from 1 and includes both ends of a range (1:3 → 1,2,3); Python counts from 0 and excludes the end (range(1,3) → 1,2). This is the most common cross-language bug.*